山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202500130002	姓名： 林毅玮	班级： 25.2
实验题目：通讯录实现		
实验学时：	实验日期： 9/24	
实验目的： 实现通讯录的功能		
软件开发环境： VS code		

1. 实验内容

基于顺序存储格式（数组描述）设计并实现一个通讯录管理系统。具体要求如下：
限制条件：不能使用 C++ STL 模板库（如 vector、string 等），必须手写线性表结构。
数据内容：通讯录中每个联系人的信息包括：姓名、电话号码、班级、宿舍。
功能要求：封装线性表类，提供插入、删除、编辑、查找（按姓名查找）等操作；并实现键盘输入一个班级，输出该班级所有成员宿舍号的异或值。
输入输出格式：根据操作码（0-插入，1-删除，2-编辑，3-查找，4-输出异或值）执行对应操作。查找操作找到输出 1，未找到输出 0；输出班级异或值时输出最终的整型异或结果

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

1.数据结构设计

Contact **结构体：**用于存储单个联系人的数据。包含四个字符数组：name[50]（姓名）、phone[20]（电话）、classId[50]（班级）、dorm[20]（宿舍）。
SequentialList **类（顺序表）**：封装了线性表的属性与操作。
私有成员：
Contact data[20000]：静态数组，用于连续存储通讯录数据（符合顺序存储结构）。
int length：记录当前线性表中实际元素的个数。
公有方法：insert（插入）、remove（删除）、edit（编辑）、search（查找）、printClassXor（班级宿舍异或统计）。

2.算法思路和复杂度分析

插入（操作码 0）：直接将新联系人追加到数组末尾（data[length]），并让 length++。**时间复杂度：**$O(1)$。
删除（操作码 1）：顺序遍历数组，通过 strcmp 匹配姓名。找到后将目标位置之后的所有元素整体向前移动一位，最后 length--。时间复杂度：$O(n)$。
编辑（操作码 2）：顺序遍历匹配姓名。找到后根据操作项 item（1-电话，2-班级，3-宿舍）使用 strcpy 更新对应字段。时间复杂度：$O(n)$。

查找（操作码 3）：顺序遍历数组匹配姓名。找到立即输出 1 并返回，遍历结束未找到则输出 0。**时间复杂度：**$O(n)$。
宿舍班级异或（操作码 4）：遍历整个数组，筛选出班级匹配的联系人。将其宿舍号字符串逐字符转换为整型数字，然后进行累加异或运算（xorResult ^= dormNum）。时间复杂度：$O(n)$。

3. 测试结果（测试输入，测试输出）

10
0 ZhangSan 13800000000 ClassA 101
0 LiSi 13900000000 ClassA 102
0 WangWu 13700000000 ClassB 201
3 ZhangSan
4 ClassA
1 ZhangSan
3 ZhangSan
2 LiSi 1 13911111111
3 LiSi
4 ClassB

1
3
0
1
201

4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

1.结果分析
程序测试结果完全符合预期。顺序存储结构（数组）在内存中是连续存放的，因此支持随机访问，遍历和查找效率较高。但在删除操作时，由于需要保持内存的连续性，必须移动大量元素，效率较低（$O(n)$）。在本次实验 $n \leq 20000$ 的数据规模下，性能完全达标。

2.遇到的问题和解决路径
在开发与调试过程中，曾遇到程序“无报错但直接结束运行”的问题，经过排查，探讨并总结了以下原因及解决途径：

问题一：多文件链接编译错误。因为将代码拆分为了 contact.h、contact.cpp 和 main.cpp，命令行编译时未同时编译两个 .cpp 文件，会导致链接器找不到函数实现（undefined reference）。
解决途径：在命令行使用 g++ main.cpp contact.cpp -o main 同时编译；在 IDE 中将所有源文件加入同一个项目构建路径。

问题二：局部大数组导致栈溢出。在 main 函数内声明 SequentialList list; 时，由于 Contact data[20000] 占用约 2.8MB 内存，而系统默认栈空间通常只有 1MB，导致程序运行时瞬间崩溃。
解决途径：将 SequentialList list; 移出 main 函数，定义为**全局变量**（存放在静态存储区），或使用 new 动态分配在堆区。

问题三：数组越界，最初代码中插入操作设定了 length >= 2005 的笔误限制，导致超过 2005 条数据

被丢弃。此外，大量输入输出未关同步流。

3. 改进和优化方向
若通讯录数据量进一步扩大（如 $n \geq 100000$），顺序表每次删除和编辑都要遍历和移动元素，效率将急剧下降。未来的改进方向：
链式存储：改用单链表或双向链表，可将插入和删除操作的时间复杂度降低至 $O(1)$（需先查找定位的情况下为 $O(n)$）。
索引优化：按姓名进行排序后使用二分查找（$O(\log n)$），或者引入哈希表（Hash Table）实现 $O(1)$级别的姓名查找。

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

```
#include <iostream>
#include <cstring> // 用于 strlen, strcpy, strcmp

using namespace std;

// 定义联系人结构体
struct Contact {
    char name[50];
    char phone[20];
    char classId[50];
    char dorm[20];
};

// 定义顺序表类
class SequentialList {
private:
    Contact data[20005]; // 静态数组，题目限制 n<=20000
    int length;              // 当前线性表长度

public:
    // 构造函数，初始化空表
    SequentialList() {
        length = 0;
    }

    // 插入操作 (对应操作码 0)
    // 题目保证输入合法，直接尾部插入
    void insert(const char* n, const char* p, const char* c, const char* d) {
        if (length >= 20005) return; // 简单的溢出保护
        strcpy(data[length].name, n);
        strcpy(data[length].phone, p);
        strcpy(data[length].classId, c);
        strcpy(data[length].dorm, d);
        length++;
```

```
}

// 删除操作 (对应操作码 1)
void remove(const char* n) {
    int index = -1;
    // 查找名字
    for (int i = 0; i < length; i++) {
        if (strcmp(data[i].name, n) == 0) {
            index = i;
            break;
        }
    }

    if (index != -1) {
        // 找到后，将后续元素前移
        for (int i = index; i < length - 1; i++) {
            data[i] = data[i + 1];
        }
        length--;
    }
}

// 编辑操作 (对应操作码 2)
// item: 1-电话, 2-班级, 3-宿舍
void edit(const char* n, int item, const char* newVal) {
    for (int i = 0; i < length; i++) {
        if (strcmp(data[i].name, n) == 0) {
            if (item == 1) strcpy(data[i].phone, newVal);
            else if (item == 2) strcpy(data[i].classId, newVal);
            else if (item == 3) strcpy(data[i].dorm, newVal);
            return; // 假设名字唯一，找到即修改并返回
        }
    }
}

// 查找操作 (对应操作码 3)
// 找到输出 1，未找到输出 0
void search(const char* n) {
    for (int i = 0; i < length; i++) {
        if (strcmp(data[i].name, n) == 0) {
            cout << 1 << endl;
            return;
        }
    }
    cout << 0 << endl;
}
```

```
    // 输出一个班所有人员信息的异或值 (对应操作码 4)
    void printClassXor(const char* c) {
        int xorResult = 0;
        bool foundAny = false; // 标记是否有该班级成员

        for (int i = 0; i < length; i++) {
            if (strcmp(data[i].classId, c) == 0) {
                // 将宿舍号字符串转换为整数进行异或
                // 假设宿舍号是纯数字，虽然题目描述是"宿舍"，这里按常规数字处理
                int dormNum = 0;
                // 简单的字符串转数字
                for (int k = 0; data[i].dorm[k] != '\0'; k++) {
                    if (data[i].dorm[k] >= '0' && data[i].dorm[k] <= '9') {
                        dormNum = dormNum * 10 + (data[i].dorm[k] - '0');
                    }
                }

                xorResult ^= dormNum;
                foundAny = true;
            }
        }

        // 题目要求输出异或值，如果没有该班级成员，异或值默认为 0
        cout << xorResult << endl;
    }
};

SequentialList list;

int main() {
    // 优化 I/O 速度
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    if (!(cin >> n)) return 0;

    int op;

    for (int i = 0; i < n; i++) {
        cin >> op;
        if (op == 0) {
            char name[50], phone[20], cls[50], dorm[20];
            cin >> name >> phone >> cls >> dorm;
            list.insert(name, phone, cls, dorm);
```

```
            }
            else if (op == 1) {
                  char name[50];
                  cin >> name;
                  list.remove(name);
            }
            else if (op == 2) {
                  char name[50], newVal[50];
                  int item;
                  cin >> name >> item >> newVal;
                  list.edit(name, item, newVal);
            }
            else if (op == 3) {
                  char name[50];
                  cin >> name;
                  list.search(name);
            }
            else if (op == 4) {
                  char cls[50];
                  cin >> cls;
                  list.printClassXor(cls);
            }
      }

      return 0;
}
```